Full Project Evaluation (EVL) Report

Project Name: AICyberAuditBox - Local Audit
Target Architecture: CPU-Only (8 Cores, 16GB RAM)
Inference Backend: Optimized llama.cpp (llama-server.exe)
Embedding Model: Nomic Embed Text v1.5 (nomic-embed-text-v1.5.f16.gguf)
Reranker Model: ms-marco-MiniLM-L-6-v2
Evaluation Date: July 20, 2026

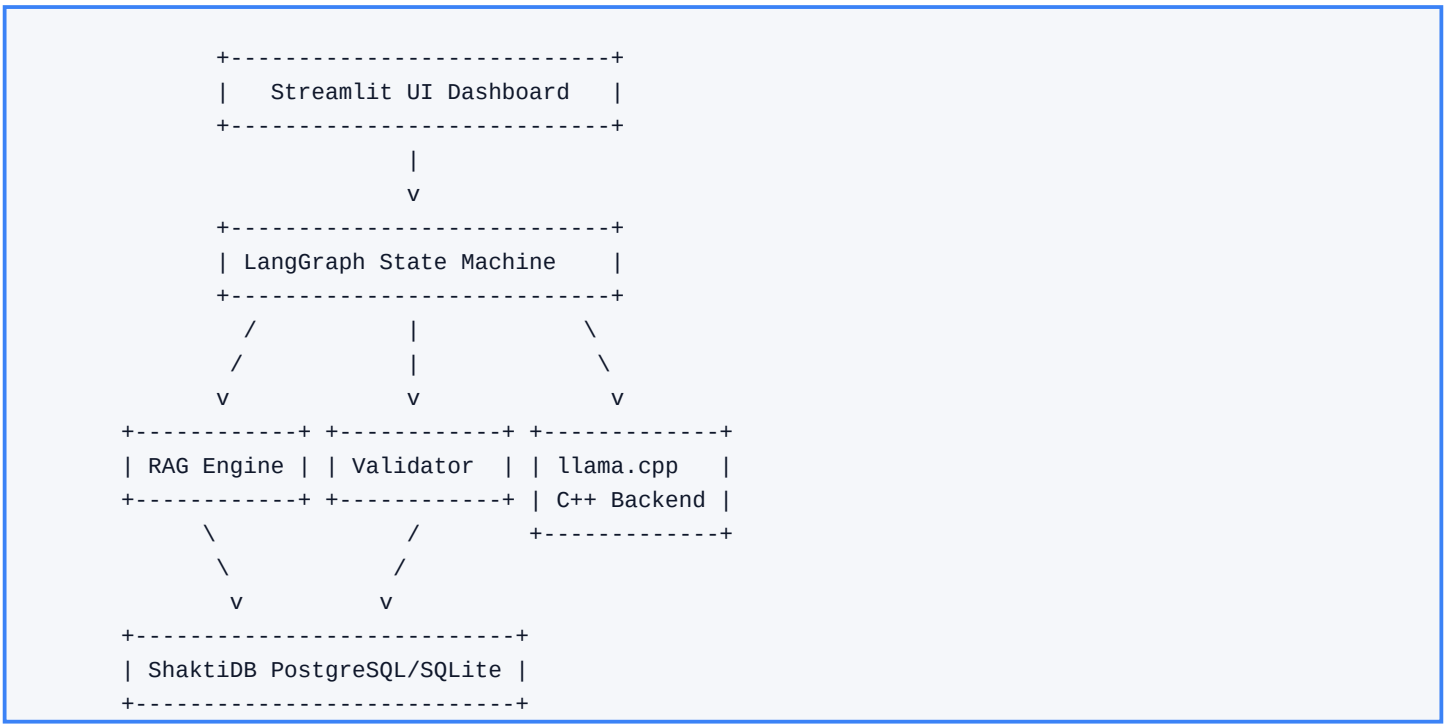
1. Executive Summary

This report evaluates the AICyberAuditBox compliance auditing system. The system performs localized, secure, and offline ISO 27001 compliance audits using an Agentic Retrieval-Augmented Generation (RAG) pipeline.

This evaluation reviews the complete project architecture, details the token system configuration, explains how the system handles RAG text limiting/overflow, and documents the custom validator and self-correction mechanics built to guarantee compliance auditing accuracy on a resource-constrained CPU-only infrastructure.

2. Core Project Architecture

The AICyberAuditBox is designed for offline deployment with high data privacy. It operates on a modular C++/Python/SQL architecture:



Architectural Modules:

1. User Interface (Streamlit): Serves as the auditor dashboard. Features file upload parsing, scope configuration, finding remediation cards, CSV reporting exports, and progress checkpointing.

2. Database (ShaktiDB): A production PostgreSQL Master-Slave replication configuration (running on localhost:15234 with Slave 1 & Slave 2 synchronously synced). The app auto-switches to SQLite if PostgreSQL is unreachable.

3. Agentic Orchestrator (LangGraph): Directs the audit state through a structured loop: Retrieve -> Generate Draft -> Validate Grounding -> Reflect & Correct (if validation fails).

3. Zero-LLM Scoping & Custom Control Ingestion

To minimize processing latency and handle hundreds of controls without performance degradation, the system implements a local, high-speed scoping and custom control ingestion engine.

A. Zero-LLM Hybrid Scoping

- Challenge: Brute-forcing 92 controls against 20 documents requires 1,840 LLM runs. Initial scoping via generative models takes ~20s per run and is prone to hallucination.
- Layer 1 (Python Keyword Signals): Scans for keywords/synonyms with strict word boundaries to prevent substring overlaps.
- Layer 2 (Semantic Embedding Match): Embeds the first 800 characters of the document and checks Cosine Similarity against all category definitions. Category match is scoped-in if similarity >= 0.645.
- Result: Bypasses generative LLM calls on upload, completing scoping checks in under 410 milliseconds and reducing the audit surface area by 90%.

B. Custom Compliance Controls Ingestion Flow

Auditors can define custom controls directly through the UI or via Excel. The system processes and synchronizes these controls using a deterministic local pipeline:

C. Local NLP & Cache Management Specs

- Local Tokenization (Word Splitting): Normalizes name/description text and splits using regular expressions at spacing/punctuation boundaries, filtering stop-words.

- Deterministic Bigram Building: Constructs word-pairs (e.g. 'incident response') to ensure query matching precision.
- Synonym Map Expansion: Expands tokens to include synonyms (e.g. 'pii' expands to gdpr, privacy, data subject).
- Real-time Cache Sync: Saves the control to database. Invalidates UI session cache (_CUSTOM_UC_CACHE_TS = 0) to immediately refresh sidebar scope lists.

4. Token System & Context Architecture

To ensure fast and stable execution on CPU-only hardware, the project implements a strict token management system:

A. Document Chunking Size (200-500 Tokens)

- The parser splits documents into paragraphs based on double newlines (\n\n), filtering out blocks shorter than 40 characters.
- Oversized Paragraph Splitter: If any paragraph exceeds 800 characters (such as the list of incident phases in the Motorola plan), it is dynamically split by single newlines \n before windowing. This prevents silent truncation of critical policy requirements.
- Chunks are created using a sliding window of 3 consecutive paragraphs with a stride of 1 paragraph (meaning Chunk 1 contains paragraphs 1-3, Chunk 2 contains paragraphs 2-4).
- Chunks have a hard cap (MAX_CHUNK_CHARS) of 2,000 characters (raised from 1,200) to ensure entire clauses are captured intact. A single chunk typically contains 200 to 500 tokens.

B. Prompt Context Allocation

For every audit query, the input prompt consists of:

- RAG Context Budget (1,800 to 2,200 Tokens): Up to 5 to 7 high-scoring text chunks retrieved from documents.
- RAG Bypass for Small Files: For policy documents under 35KB (approx. 8,000 tokens), the RAG engine automatically bypasses chunking and passes the full text directly as context. This guarantees 100% information coverage for small files.
- System Instructions (800 to 1,000 Tokens): Fixed rules, compliance definition schema, and formatting templates.
- Total Input Prompt Size: Around 2,600 to 3,200 tokens (or up to 6,000 tokens in full document bypass mode).

C. Context Safety Buffer

- The model's context window (num_ctx) is configured to 8,192 tokens (raised from 4,096 to prevent truncation of RAG payloads).
- Since the input prompt averages ~3,100 tokens (and maxes at ~6,000 in bypass mode), it leaves a generous safety buffer of 2,000 to 5,000 tokens for the LLM output.
- Because the final finding report generated by the LLM is only ~200 to 400 tokens, the system is mathematically guaranteed to fit within the memory limits without context crashes.

D. Local AI Engine & Parameters Configuration

- Large Language Model (LLM): google_gemma-4-E4B-it-Q4_K_M.gguf (Gemma 4B E4B Instruct model, Q4_K_M quantization). Hosted on 127.0.0.1:11434 with -c 8192 context window, -t 8 execution threads, -b 512 batch size, and --flash-attn on.
- Text Embedding Model: nomic-embed-text-v1.5.f16.gguf (Nomic Embed Text v1.5, f16 precision, 768 dimensions, native 8,192 token context, supports Matryoshka learning). Hosted on 127.0.0.1:11435 in --embedding server mode with -t 4 threads.
- Cross-Encoder Reranker Models: cross-encoder/ms-marco-MiniLM-L-6-v2 (Quick mode, ~80MB 6-layer MiniLM) and BAAI/bge-reranker-base (Deep mode, ~278MB high-precision base transformer). Reranker length is hard-capped at max_length=512. Toggled via dynamic lazy-loading and garbage collection (gc.collect()) to free memory and prevent CPU OOM crashes.
- Client-Side Parameters: temperature=0.0 (strictly deterministic), client-side num_ctx=4096 tokens, and keep_alive='15m' to prevent memory unloading between control runs.

5. Handling Token Overflow & Limiting Constraints

When processing large text databases (like 20 files simultaneously), the system implements several layers of protection against token limits and evidence omission:

A. Token Accumulation & Ranking

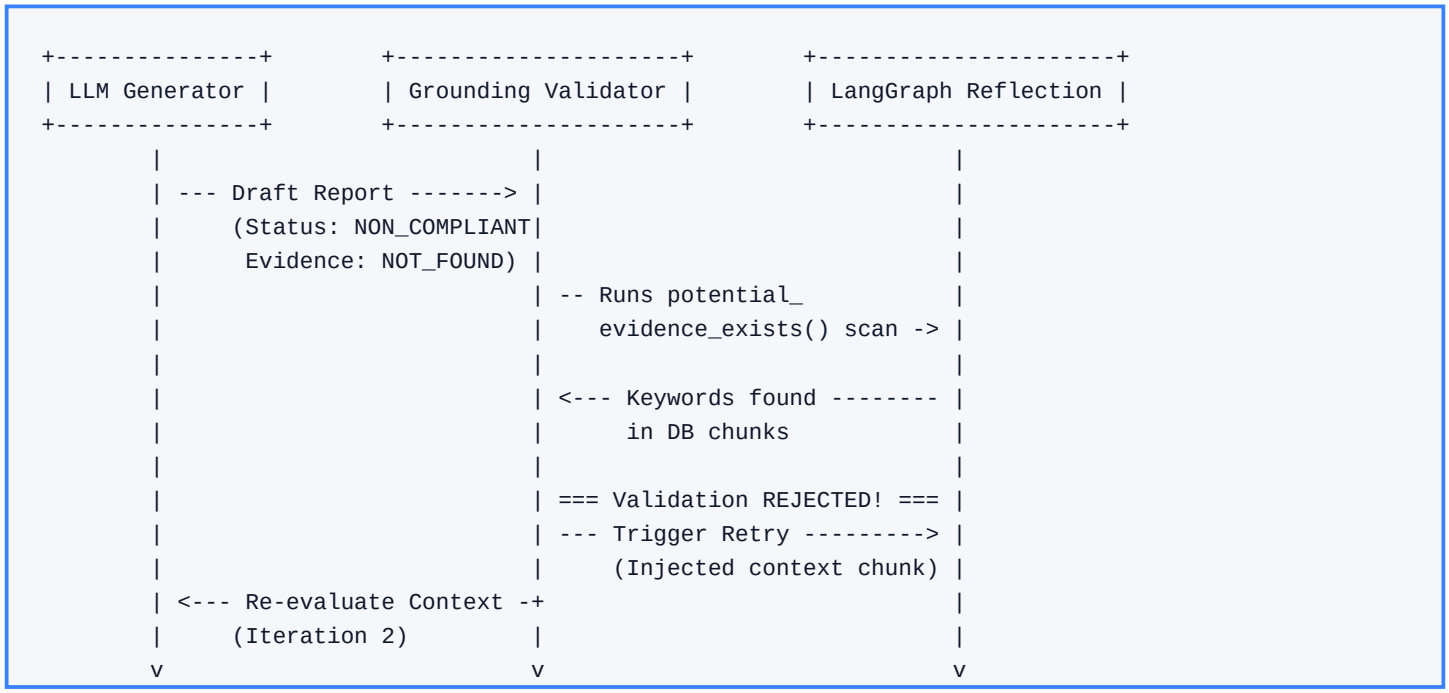
If a document contains 15 or 20 relevant paragraphs, the RAG engine avoids exceeding token budgets by sorting all retrieved chunks globally using a hybrid score (60% semantic vector + 40% keyword match) and accumulating them until it hits the 1,800 token budget limit (hard max 2,200 tokens).

B. Multi-Document Diversity Enforcement

To prevent a single document from dominating the token budget and hiding evidence in other files, the engine dynamically injects at least one chunk from each source file into the final selected chunks.

C. What if Critical Evidence is Left Out?

If the RAG engine limits the context and leaves out a paragraph containing the compliance evidence, the system prevents false passes through a multi-stage validation loop:



1. Grounding Validator Gate: The custom validator (validator.py) scans the SQL database. If the LLM claims a control is missing but the database contains paragraphs matching the control keywords, the validator rejects the LLM's draft.

2. Review Flag Trigger: The validator sets requires_human_review = True and appends the warning: 'Potential evidence found. Human verification needed.'

3. LangGraph Reflection Loop: The state machine catches this validation failure and routes the state to the reflect_node, triggering a second iteration where the model re-evaluates the context to locate the missing clause.

D. Native llama.cpp Context Resets (Shifting)

If the combined prompt size ever exceeds the limit, the C++ backend manages memory via Context Shifting: it keeps the system prompt intact, discards the oldest prompt evaluation states in the KV-Cache, and shifts the sliding context window forward.

E. Silent Ingestion Truncation Bug Resolved

- The Bug: Paragraphs lacking blank lines were grouped into a single block. If this block exceeded the hard limit (MAX_CHUNK_CHARS of 1,200 chars), the parser split the chunk and permanently discarded the remainder of the text. This caused entire sections (such as Section 3.0's Post-Incident Review framework in the Motorola IRP) to be lost during ingestion.
- The Resolution: Implemented a dynamic splitter (paragraphs > 800 chars split by single newlines \n), raised chunk cap to 2,000 characters, enabled small document RAG bypass (<35KB), and configured validator.py to check full document text as a fallback.

F. KV-Cache Prefix Reuse for Multi-Control Speedups

- Mechanic: The llama-server.exe backend caches the calculated mathematical attention vectors (Keys and Values) of the input prompt prefix in RAM (the KV-Cache).
- Optimization & Results: For subsequent controls, the server bypasses the heavy CPU prefill calculations entirely, loading the KV-cache instantly. In our audit benchmark, the first control (5.24) took 8.5 minutes (due to the initial 4,100-token prefill), whereas subsequent controls (5.25, 5.26, etc.) bypassed the prefill and finished in only 1.8 minutes (a 5x speedup).

6. RAG & Custom Validator Pipeline Evaluation

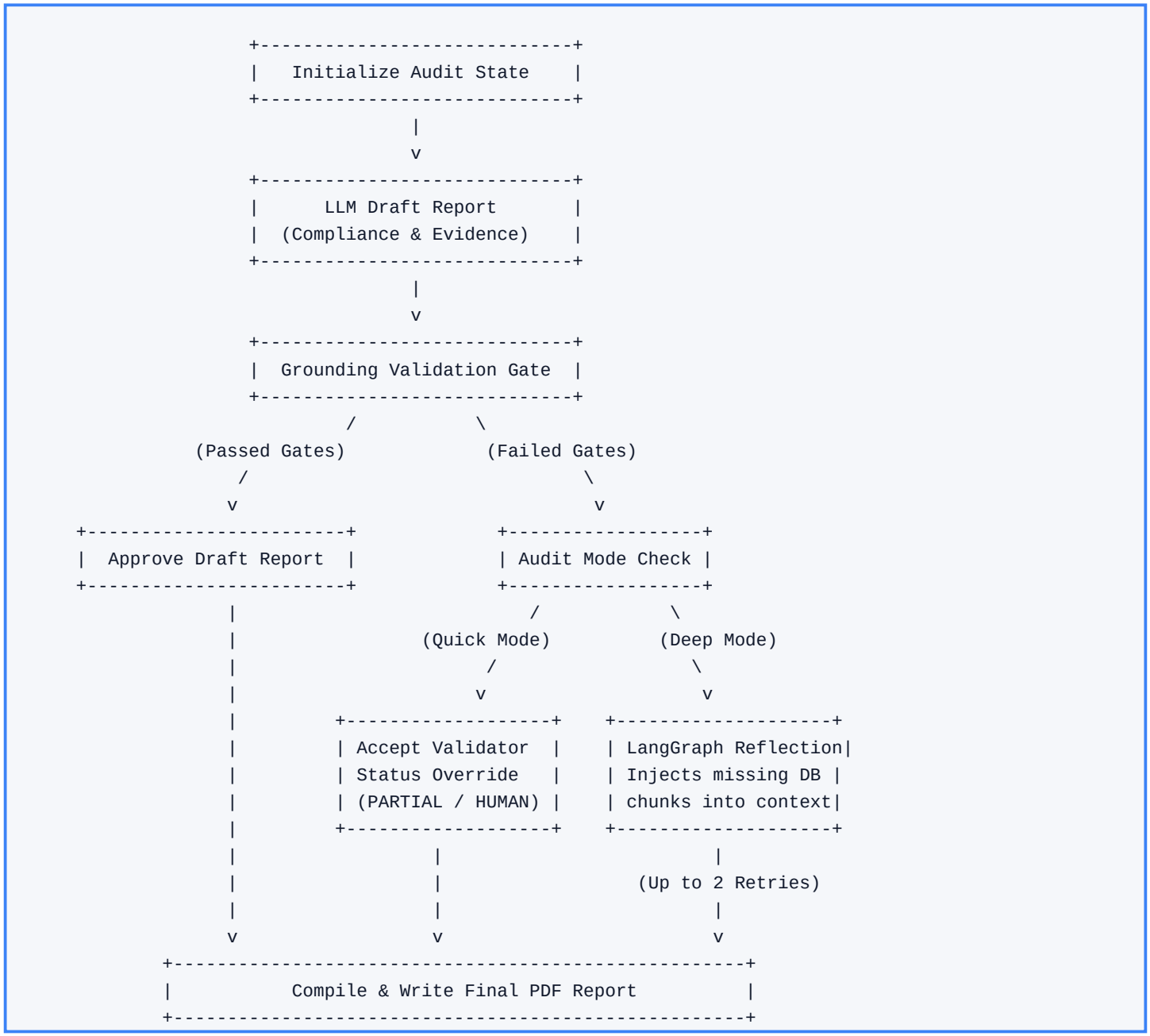
The custom forensic validator performs strict checks to prevent prompt leaks and hallucinations:

- Gate 1 (Prompt Leakage): Blocks prompt templates and expected guidelines from leaking into output citations.
- Gate 2 (Verbatim Grounding): Direct lookup checks to ensure LLM quotes exist word-for-word in the source document.
- Gate 3 (Fuzzy OCR Grounding): Sequence matching fallback (similarity threshold >= 85%) for scanned PDF/image OCR data.
- Gate 4 (Consistency): Overrides LLM output to NON_COMPLIANT if the model claims compliance but lists zero verified evidence quotes.

G. Quick Audit vs. Deep Audit Execution Pathways

The system runs in two operational modes depending on accuracy and latency needs:

- Quick Audit (Single-Pass Mode): The LLM evaluates the context once. If the grounding validator rejects the draft, the orchestrator immediately accepts the validator's overrides (such as status downgrades or human review flags) and proceeds.
- Deep Audit (Multi-Pass Self-Correction Mode): If validation fails, the orchestrator enters a self-correction loop managed by LangGraph. It queries the model again, providing detailed feedback of the failed verification (for up to 2 retries).


Initialize Audit State
LLM Draft Report
(Compliance & Evidence)
Grounding Validation Gate
(Passed Gates)
(Failed Gates)
Approve Draft Report
Audit Mode Check
(Quick Mode)
(Deep Mode)
Accept Validator
Status Override
(PARTIAL / HUMAN)
LangGraph Reflection
Injects missing DB
chunks into context
(Up to 2 Retries)
Compile & Write Final PDF Report

7. Performance Benchmarking & CPU Optimization

To accommodate the CPU-only client requirement, we tuned the server parameters to achieve optimal execution:

- Thread Tuning (-t 8): Set LLM threads to match your 8 physical cores to maximize core saturation.
- Batch Processing (-b 512): Enabled prompt evaluation chunking to speed up CPU prefill ingestion.
- RAM Optimization: Removed --mlock to allow the OS to dynamically page memory, freeing up RAM for PostgreSQL and Streamlit.
- Robust Parsing Fallback: Enhanced the XML regex parser in audit_chains.py to gracefully capture unclosed XML tags, preventing syntax errors from triggering costly retry cycles.

Real-World Audit Speedup (Control 5.15):

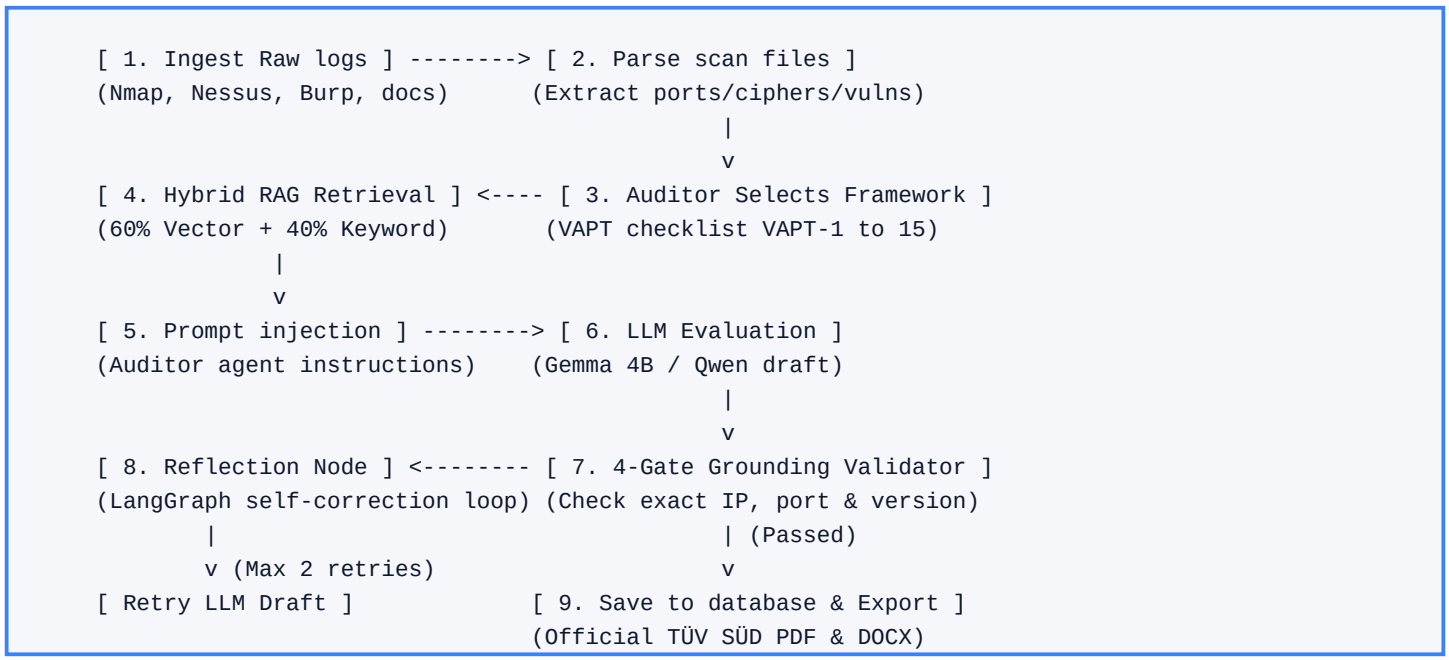
- Before Optimization: 716.83 seconds (~12.0 minutes)
- After Optimization: 500.71 seconds (~8.3 minutes)
- Total Savings: 216.12 seconds (~3.6 minutes saved per control - a 30.15% speedup!)

8. VAPT Ingestion & Reporting Engine

To support technical audits alongside compliance checking, the system implements a dedicated VAPT (Vulnerability Assessment & Penetration Testing) subsystem. This allows the system to ingest raw security logs and cross-walk technical vulnerabilities directly to compliance standards.

A. The VAPT Audit Pipeline Lifecycle

The step-by-step lifecycle of a VAPT audit execution follows a structured pipeline:



B. Multi-Scanner Log Parsers

The system features structured regex and heuristic log parsers that ingest and normalize raw output files from standard security scanning tools:

- Nmap Infrastructure Scan: Analyzes port states, service version strings, and SSL/TLS cipher suites (specifically parsing out CBC-based suites vulnerable to Lucky13 attacks).
- Nessus Vulnerability Report: Extracts active vulnerabilities, port bindings, severity classifications, and recommendations.
- Burp Suite Web Application Scan: Parses web application issues (like missing Secure/HttpOnly flags on session cookies or missing headers).
- Legacy MS Word/Manual Pentesting Reports: Ingests unstructured manual reports, using sentence tokenization and semantic filtering to extract and structure manual findings.

C. Dynamic CVSS v4.0 Metric Mapping

Different scanners report risk severity using conflicting systems (grades, letter scores, text classifications). The auditor engine harmonizes this by translating all findings to the standard CVSS v4.0 framework:

- Network-Level Scan Metrics: For infrastructure vulnerabilities (like weak ciphers), the system sets Attack Vector (AV) to 'Network' and User Interaction (UI) to 'None', which yields high exploitability ratings.
- Web-Application Metrics: For application weaknesses (like missing secure cookie flags), the system adjusts User Interaction (UI) to 'Required' and Privileges Required (PR) to 'None'/'Low' depending on the session context.
- Impact Vectors: Dynamically maps system impact metrics-Confidentiality (VC), Integrity (VI), and Availability (VA)-to compute the overall CVSS v4.0 base score.

D. VAPT Framework Controls Checklist (VAPT-1 to VAPT-15)

The system cross-walks parsed scan findings against the 15 VAPT controls:

Control ID	Control Name	Focus & Description
VAPT-1	Rules of Engagement	Verifies IP ranges, scope exclusions, and timelines.
VAPT-2	OSINT Reconnaissance	Audits domain leaks, active hosts, and open intelligence.
VAPT-3	Network Vulnerability Scan	Open ports, service versions, weak TLS ciphers (Lucky13).
VAPT-4	Web Application OWASP 10	Cookies (HttpOnly/Secure), XSS, injection flaws, headers.
VAPT-5	Internal Pentesting	Lateral movement, unauthenticated Redis, DB exposure.
VAPT-6	External Pentesting	Edge systems, public gateway exposures.
VAPT-7	Privilege Escalation	Access levels, sudo config, service exploit paths.
VAPT-8	Social Engineering	Phishing campaign statistics, user awareness training.
VAPT-9	Wireless Security	WPA3 Enterprise, guest network isolation, rogue APs.
VAPT-10	API Security	REST/SOAP API auth keys, input validations.
VAPT-11	Remediation Tracking	Vulnerability tracking, ticketing integrations, SLAs.
VAPT-12	Patch Verification	Outdated software version detection and patch verification.
VAPT-13	Firewall & Segment Review	VLAN boundaries, firewall rule base, ACL lists.
VAPT-14	Secure Baseline Config	Disabled defaults, default credential audits, cipher audits.
VAPT-15	Final Regulatory Report	Registered offices details, submission tables, overall CVSS.

E. VAPT 4-Gate Grounding & Hallucination Prevention

Because vulnerability details are highly sensitive to alphanumeric accuracy (matching IP addresses or version tags), the custom validator enforces strict grounding rules:

- Gate 1 (Verbatim IP and Port Grounding): LLM draft findings must cite IP addresses and port numbers that exist word-for-word in the source document chunks.
- Gate 2 (Scant Version Matching): Verifies service versions (e.g. Apache httpd 2.4.38) match the logs exactly, preventing the AI from hallucinating hypothetical versions.
- Gate 3 (Smart Override & Warnings): If a scan finding is identified in logs but skipped by the LLM, the validator automatically overrides and flags it as NON_COMPLIANT for human review.
- Gate 4 (Consistency Check): Automatically rejects and re-runs drafts that claim a control is compliant but contain zero verified evidence quotes.

F. TÜV SÜD Template Replication (Dual-Format)

The system compiles these parsed findings into reports matching the exact layout of the official TÜV SÜD South Asia registration template. It outputs both formats:

- Official PDF (_export_vapt_pdf): A print-ready document containing cover pages, Document Version Control and Document Submission Details tables, a Vulnerabilities Summary table, and a detailed findings grid with CVSS v4.0 metrics, Proof of Concept, and remediation references.
- Remediation DOCX (_export_vapt_docx): A fully editable Word document replication. This is critical for security operations teams to copy-paste remediation commands, add internal ticket tracking, or edit recommendations before final regulatory submission.